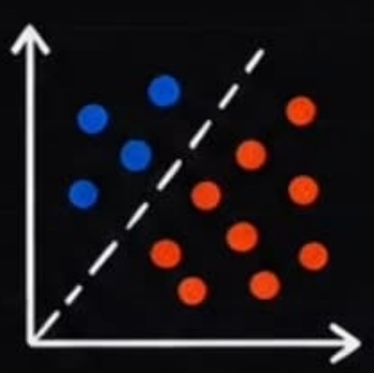
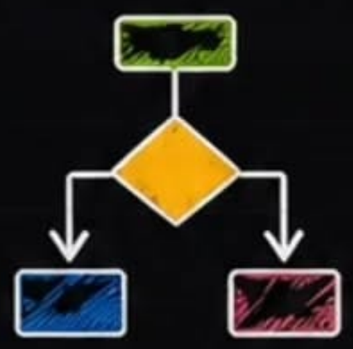
SRIPADH K

MACHINE LEARNING

COMPLETE NOTES



Algorithms & Concepts



Models & Training

Supervised & Unsupervised Learning

Evaluation & Metrics

Practical & Projects

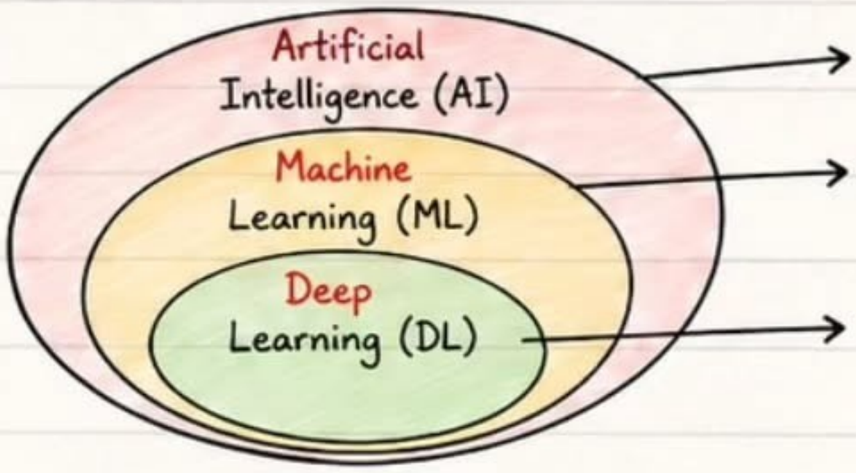
BEGINNER TO ADVANCED

SWIPE TO NEXT

Machine Learning Fundamentals

1 AI vs ML vs Deep Learning

Hierarchical relationship –
DL is a subset of ML,
ML is a subset of AI.



2 What is Machine Learning?

ML is the field of study that gives computers the ability to learn from data and make predictions or decisions without being explicitly programmed.

> ★ **Key Idea**
>
> Learn patterns from data → Generalize → Make predictions on new, unseen data.

3 Traditional Programming vs Machine Learning

Traditional Programming

Input (Data) → Program (Rules) → Output (Result)

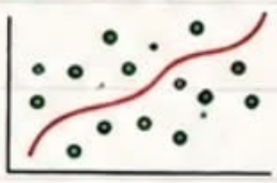
Rules are written by humans.
Works well when rules are known.

Machine Learning

Input (Data) → ML Model (Learning) → Output (Prediction)

Model learns rules/patterns from data.
Works well when rules are unknown or complex.

4 How Does a Machine Learn?

① Collect Data	② Prepare Data	③ Train Model	④ Evaluate Model	⑤ Make Predictions
				
Gather relevant and quality data.	Clean, handle missing values, encode, scale, feature engineering.	Model learns patterns from training data.	Check performance on validation/test data.	Use model to predict new, unseen data.

5 Real-World Applications

Finance

Fraud detection, credit scoring, algorithmic trading.

E-commerce

Recommendation systems, customer segmentation, demand forecasting.

Healthcare

Disease prediction, medical imaging, patient risk scoring.

Automotive

Autonomous driving, predictive maintenance, route optimization.

Communication

Spam detection, sentiment analysis, chatbots.

Industry

Quality inspection, fault detection, process optimization.

> ✔ **Remember**
>
> - ML learns patterns, not hard-coded rules.
> - Quality data > Complex models.
> - Good ML = Good data + Good problem framing.

> 💡 **Interview Tip**
>
> Always start by understanding the problem, the data and the goal. Model is just one part of the solution!

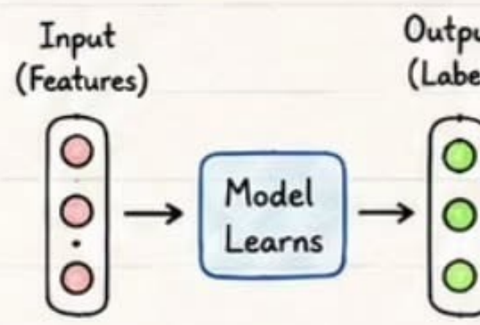
Types of Machine Learning

Machine Learning (ML) algorithms learn patterns from data.
Based on the type of data & feedback, ML can be broadly classified into 4 types.

1 Supervised Learning

Model learns from labeled data.

We know the correct output (target).



Examples

- Email Spam Detection
- House Price Prediction
- Credit Risk Scoring
- Image Classification

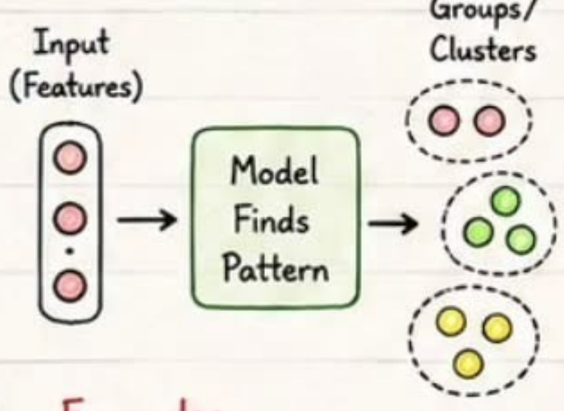
When to Use?

Use when we have labeled historical data and want to predict a specific output.

2 Unsupervised Learning

Model finds patterns or structure in unlabeled data.

No correct output is given.



Examples

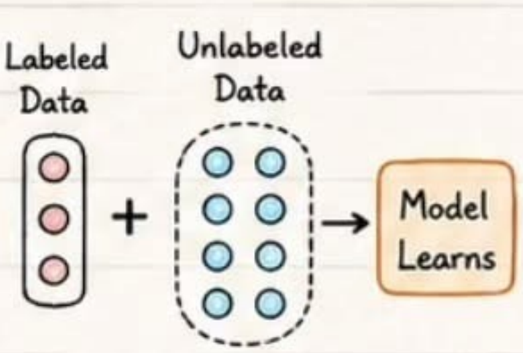
- Customer Segmentation
- Market Basket Analysis
- Topic Modeling
- Anomaly Detection

When to Use?

Use when data is not labeled and we want to explore structure or grouping.

3 Semi-Supervised Learning

Combines a small amount of labeled data with a large amount of unlabeled data.



Examples

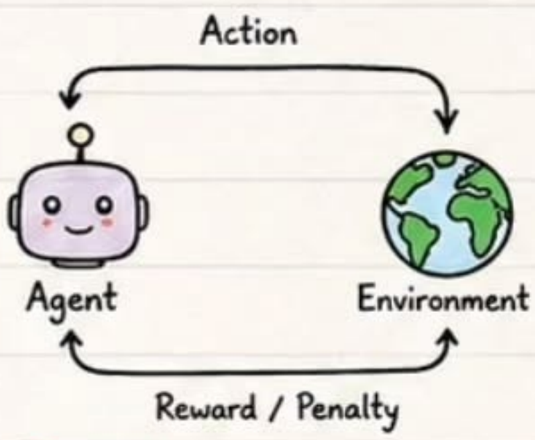
- Image Classification
- Speech Recognition
- Medical Diagnosis
- Text Classification

When to Use?

Use when labeled data is expensive or hard to get, but unlabeled data is abundant.

4 Reinforcement Learning

Model (agent) learns by interacting with environment and receives rewards or penalties.



Examples

- Game Playing (Atari, Chess)
- Robot Navigation
- Self-driving Cars
- Recommendation Systems

When to Use?

Use when there is a sequential decision process and feedback is given in the form of rewards.

Quick Comparison

Type	Data	Labels	Goal	Examples	Best For
Supervised Learning	Labeled Data	✓	Predict output	Spam Detection, House Price	Prediction & classification tasks
Unsupervised Learning	Unlabeled Data	✗	Find patterns/ structure	Customer Segmentation, Anomaly Detection	Exploratory data analysis
Semi-Supervised Learning	Labeled + Unlabeled Data	Partial	Improve learning with less label	Image Classification, Speech Recognition	When labeled data is limited
Reinforcement Learning	Interaction Data	No	Maximize cumulative reward	Game Playing, Robotics	Sequential decision making problems

Key Takeaways

- ✓ Supervised → Predict
- ✓ Unsupervised → Discover
- ✓ Semi-Supervised → Leverage limited labels
- ✓ Reinforcement → Learn by trial & reward

✓ Remember

Choose the right type based on your data, problem, and goal!

⚠ Common Mistake

Using supervised algorithms on unlabeled data.

Interview Tip

Clearly state the type of ML, why you chose it, and how it fits your problem.

Data & ML Foundations

High quality data is the foundation of every successful ML model.
Let's understand the key building blocks.

① Key Terms

- **Dataset** : Collection of data entries.
- **Data Point / Sample** : Single record in a dataset.
- **Feature** : Input variable (X) used to describe patterns.
- **Label** : Output value (Y) associated with a data point.
- **Target Variable** : The label we want the model to predict.

Example Dataset (Customer Churn)

ID	Age	Tenure (months)	Monthly Charges ($)	Contract Type	Churn (Target)
1	28	12	29.50	Month-to-month	1
2	45	24	56.70	One year	0
3	34	6	25.10	Month-to-month	1
...	...	...	...	...	...

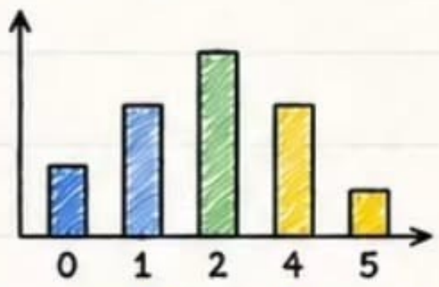
Features (X) | Label (Y)

② Types of Data

Numerical Data

Values are numbers.

- Discrete (countable)
- Continuous (measurements)



(e.g., Age, Income, Temperature)

Categorical Data

Values are categories or labels.

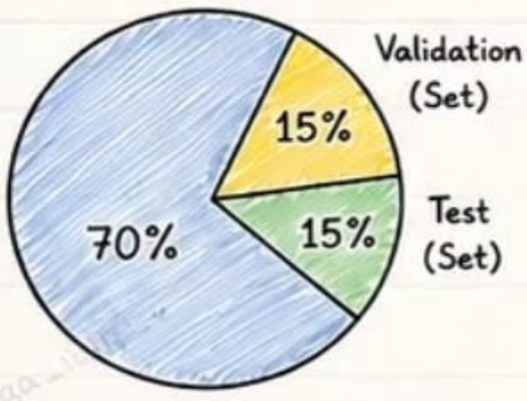
- Nominal (no order)
- Ordinal (with order)

Nominal: Red | Blue | Green

Ordinal: Low | Medium | High

(e.g., City, Gender, Education Level)

③ Train / Validation / Test Split



- **Training Set (70%)**
  Used to train the model. Model learns patterns from this data.
- **Validation Set (15%)**
  Used to tune hyperparameters and choose the best model.
- **Test Set (15%)**
  Used only once to evaluate the final model performance on unseen data.

Why Split?

- ✓ Helps prevent Overfitting
- ✓ Ensures model generalizes well
- ✓ Gives unbiased evaluation on unseen data

④ Data Point Anatomy

Feature 1	Feature 2	...	Feature n	Target (Label)
Age = 30	Income = 45K	...	Tenure = 12	Churn = 1

Input Features (X)
Help model understand patterns.

Target / Label (Y)
What model should predict.

⑤ Real-World Example

Customer Churn Prediction

Predict whether a customer will leave the service.

- Features: Age, Tenure, Usage, Monthly Charges, Contract Type, ...
- Target: Churn (0 = No, 1 = Yes)

☆ Interview Tip

Good data >> Good model.
Most ML projects spend 70-80% of time on data preparation.

✓ Best Practices

- Understand data before modeling.
- Handle missing values and outliers.
- Avoid Data Leakage at all costs.

⚠ Common Mistakes

- Using test set during training.
- Ignoring missing values.
- Building model on unclean data.

SRIPADH K 4

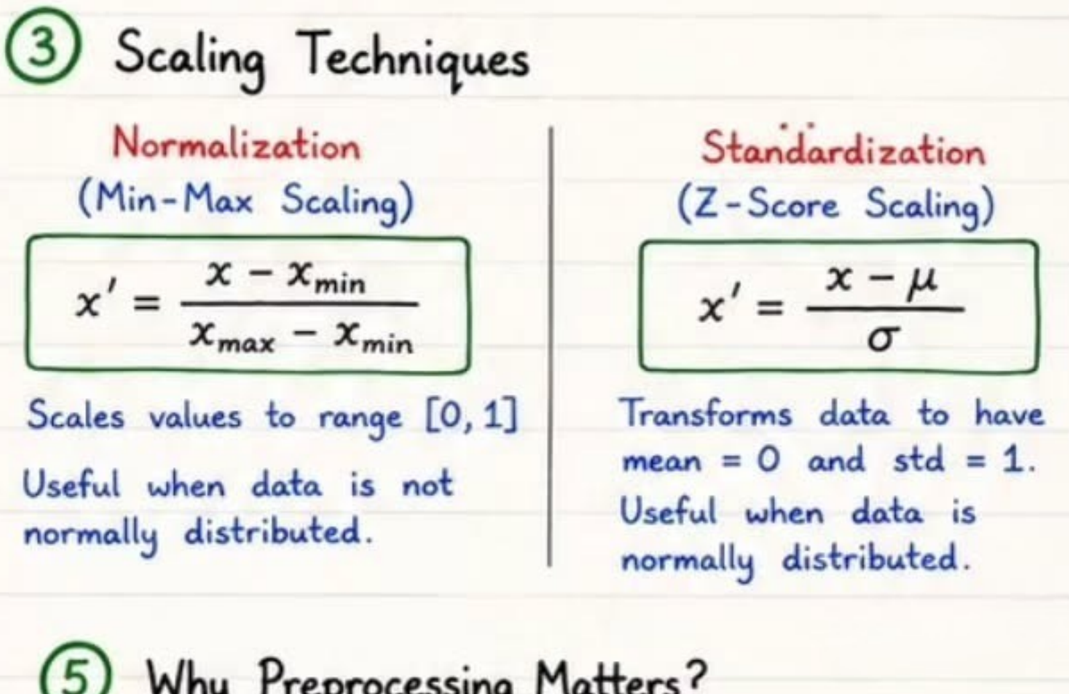
Data Preprocessing

Data preprocessing is the process of cleaning and preparing raw data so that it can be used to train effective Machine Learning models.

① Key Steps

- **Missing Values**
  Handle missing or null values using strategies like mean, median, mode or drop.
- **Duplicates**
  Remove duplicate rows to avoid biased results.
- **Outliers**
  Detect and handle extreme values using IQR, Z-Score or domain knowledge.
- **Categorical Encoding**
  Convert categorical data into numerical form. (Label Encoding / One-Hot Encoding)
- **Feature Scaling**
  Scale features to same range for better model performance. (Normalization / Standardization)
- **Train-Test Split**
  Split data into training, validation and test sets before model training.
- **Data Leakage**
  Prevent information from test data leaking into training process.

② Train-Validation-Test Split



Tip: Always split the data before any preprocessing to avoid data leakage.

③ Scaling Techniques

Normalization (Min-Max Scaling)

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Scales values to range [0, 1]

Useful when data is not normally distributed.

Standardization (Z-Score Scaling)

$$x' = \frac{x - \mu}{\sigma}$$

Transforms data to have mean = 0 and std = 1.

Useful when data is normally distributed.

④ Python Example

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Load data
df = pd.read_csv('data.csv')
# Handle missing values
df.fillna(df.mean(numeric_only=True), inplace=True)
# One-Hot Encoding
df = pd.get_dummies(df, drop_first=True)
# Features and Target
X = df.drop('target', axis=1)    y = df['target']
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)
# Feature Scaling (Standardization)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

Always fit the scaler on training data and transform the test data using the same scaler.

⑤ Why Preprocessing Matters?

- ✓ Improves model accuracy and stability.
- ✓ Speeds up training and convergence.
- ✓ Helps algorithms learn meaningful patterns.
- ✓ Prevents overfitting due to noisy data.
- ✓ Essential step in building reliable ML models.

⚠ Common Mistakes

- ✗ Applying preprocessing before splitting the data.
- ✗ Not handling outliers and missing values.
- ✗ Using test data information in training.
- ✗ Incorrect encodting of categorical variables.
- ✗ Not scaling features for distance-based models.

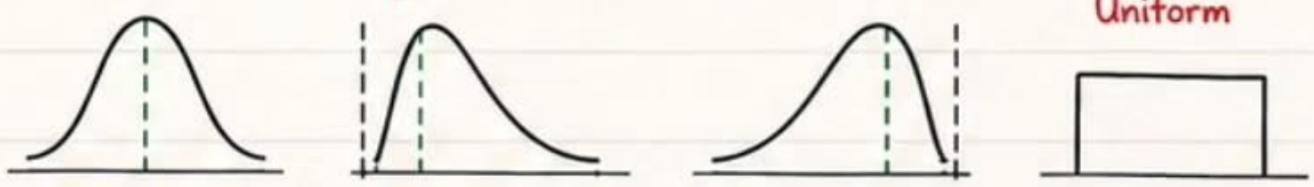
Exploratory Data Analysis (EDA)

EDA is the process of exploring and understanding data to discover patterns, check quality, test assumptions and extract useful insights before building models.

1 Key Statistical Measures

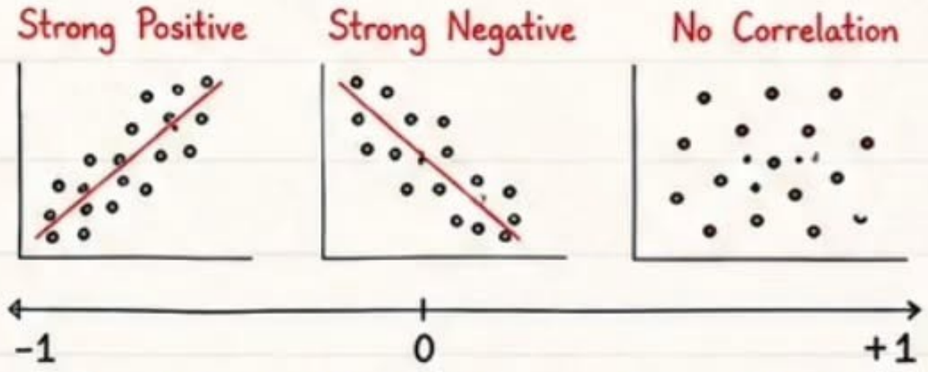
Measure	Formula / Meaning
Mean ($\bar{x}$)	$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ Average of all values.
Median	Middle value when data is sorted.
Mode	Most frequently occurring value.
Variance (σ^2)	$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ Average squared deviation.
Std Deviation(σ)	$\sigma = \sqrt{\sigma^2}$ Spread of data.

2 Distributions (Understand Data Shape)



Symmetric, bell shaped. | Long tail on right side. | Long tail on left side. | All values are equally likely.

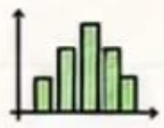
3 Correlation (Relationship between Features)



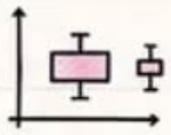
- r > 0 → Positive
- r < 0 → Negative
- r ≈ 0 → No linear relationship

Correlation Coefficient (r) ranges from −1 to +1

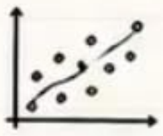
4 Visualization Techniques



Histogram
Shows frequency distribution of continuous data.



Box Plot
Shows median, quartiles, spread and outliers.



Scatter Plot
Shows relationship between two numerical features.

Heatmap
Visualizes correlation between multiple features.

Pie Chart
Shows proportion of categories.

5 Patterns & Anomalies

Patterns

- ✓ Trends (increase/decrease over time)
- ✓ Seasonality (regular repeating pattern)
- ✓ Clusters (natural groupings)
- ✓ Relationships (feature interactions)

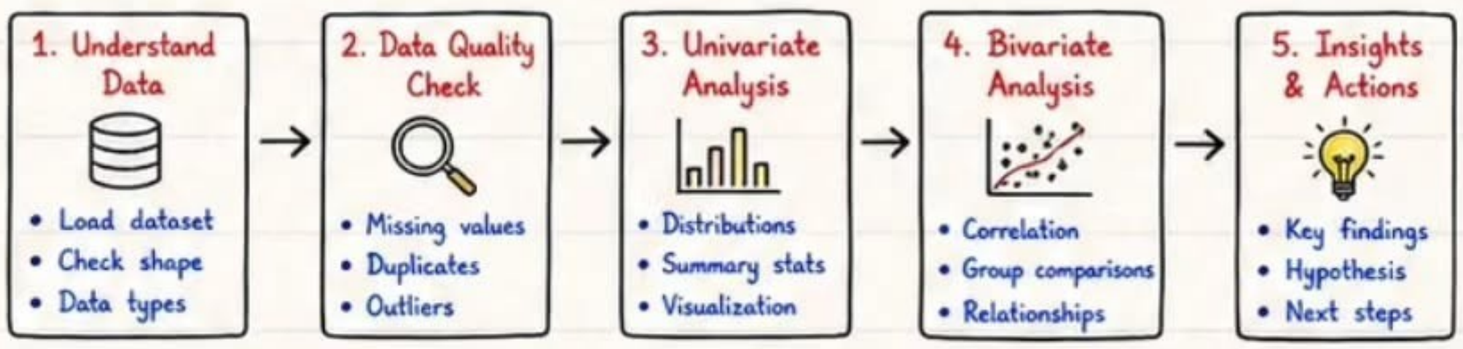
Anomalies / Outliers

- ✗ Values far from normal range
- ✗ May indicate errors or rare events
- ✗ Detect using IQR, Z-Score
- ✗ Handle with domain knowledge

Remember

Outliers are not always bad. They can be valuable insights!

6 EDA Workflow



✓ Best Practices

- ✓ Understand your data before modeling.
- ✓ Use visualizations along with statistics.
- ✓ Handle missing values & outliers carefully.
- ✓ Document key insights and assumptions.
- ✓ Prevent data leakage at every step.

Interview Tip

- ☆ Always explain what the data is telling you.
- ☆ Mention tools: Pandas, NumPy, Matplotlib, Seaborn, Profiling (ydata-profiling).
- ☆ Show how EDA helped in better modeling.

✗ Common Mistakes

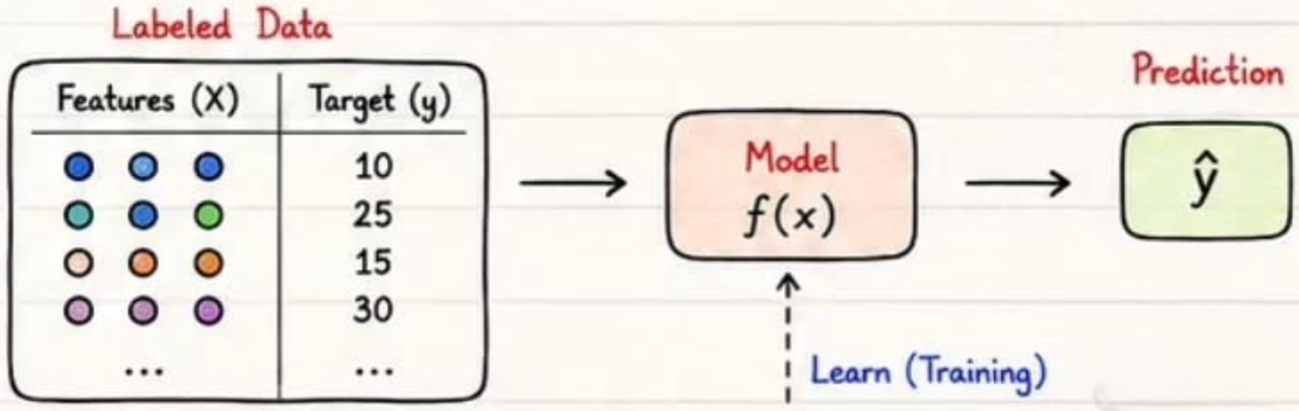
- ✗ Skipping EDA and jumping to modeling.
- ✗ Ignoring missing values & outliers.
- ✗ Relying only on accuracy for evaluation.
- ✗ Not checking data leakage.
- ✗ Misinterpreting correlation as causation.

Supervised Learning

In Supervised Learning, we learn a mapping from input (features) to output (target) using labeled data.

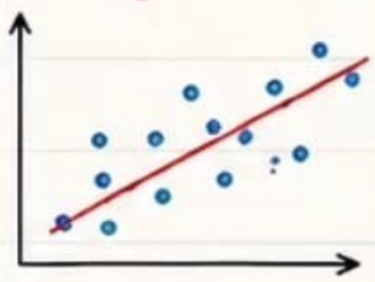
1 What is Supervised Learning?

- Uses labeled data (features + target).
- Learns a function $f(x)$ that maps inputs to the correct output.
- Goal: Generalize from known examples to unseen data.



2 Regression vs Classification

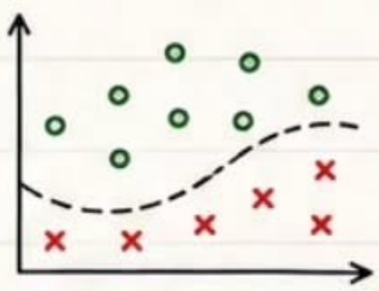
Regression (Continuous Output)



Predicts a continuous numeric value.

Examples:
- House price prediction
- Temperature forecasting
- Sales prediction

Classification (Discrete Output)



Predicts a discrete class or category.

Examples:
- Spam vs Not Spam
- Fraud vs Legit
- Cat vs Dog

Key Difference

Regression → "How much?" (Continuous value)

Classification → "Which class?" (Discrete label)

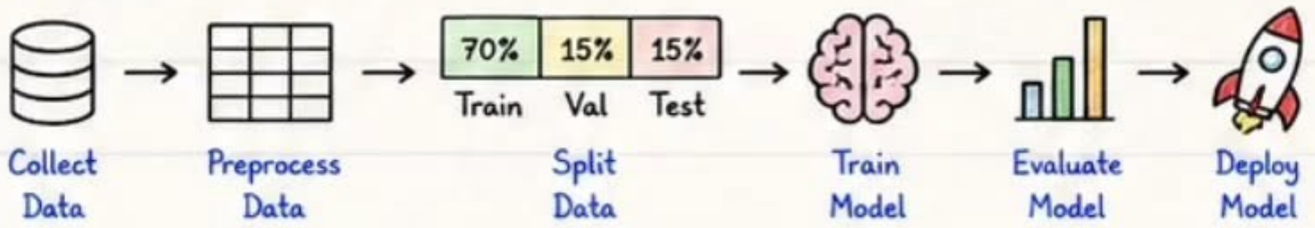
Target Variable (y)

- Regression → Numeric (e.g., 3.14, 100, -5)
- Classification → Categorical (e.g., 0/1, Yes/No, A/B)

3 Problem Formulation

- Define Objective : What are we trying to predict?
- Collect Data : Gather relevant data with correct labels.
- Define Features : Choose or engineer informative features.
- Define Target : Decide target variable (y).
- Split Data : Train / Validation / Test.
- Choose Metric : Select evaluation metric.
- Select Algorithm : Pick suitable algorithms.
- Train Model : Learn patterns from training data.
- Evaluate Model : Validate & test performance.
- Deploy Model : Use model for real-world prediction.

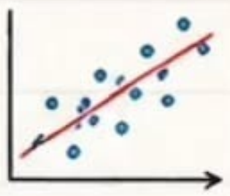
4 Typical Workflow



Tip: Always prevent data leakage! Information from test data must never be used during training.

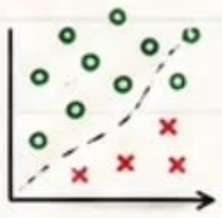
5 Common Algorithms

Regression Algorithms



- ✓ Linear Regression
- ✓ Ridge Regression
- ✓ Lasso Regression
- ✓ Polynomial Regression
- ✓ Decision Tree Regressor
- ✓ Random Forest Regressor
- ✓ Gradient Boosting (XGBoost, LightGBM, CatBoost)

Classification Algorithms



- ✓ Logistic Regression
- ✓ K-Nearest Neighbors (KNN)
- ✓ Naive Bayes
- ✓ Decision Tree Classifier
- ✓ Random Forest Classifier
- ✓ Support Vector Machine (SVM)
- ✓ Gradient Boosting (XGBoost, LightGBM, CatBoost)

6 Real-World Applications

- Regression → House Price Prediction
- Regression → Sales Forecasting
- Regression → Weather Prediction
- Classification → Spam Detection
- Classification → Fraud Detection
- Classification → Image Classification

Remember

Better data + Good features + Right model = Accurate predictions!

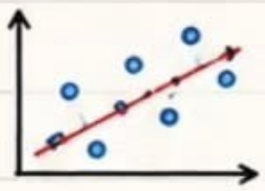
Interview Tip

Be ready to explain when to use Regression vs Classification with examples.

Common Mistakes

- ✗ Wrong target variable
- ✗ Data leakage
- ✗ Using accuracy for regression
- ✗ Skipping validation
- ✗ Ignoring class imbalance (classification)

SRIPADH K

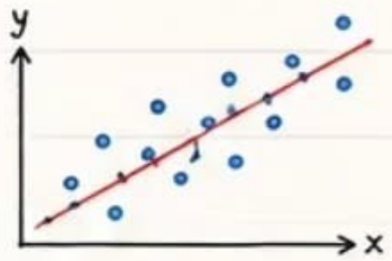


Regression Algorithms

Regression predicts a continuous numeric value. We model the relationship between features (X) and target (y).

① Types of Regression Algorithms

Linear Regression



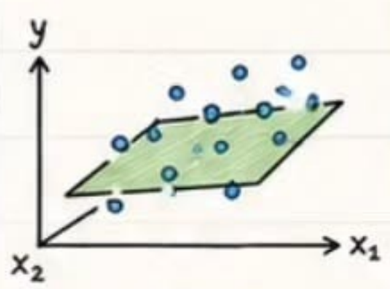
Models linear relationship between X and y.

Equation:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Use Case:
Price prediction, Sales forecasting, etc.

Multiple Regression



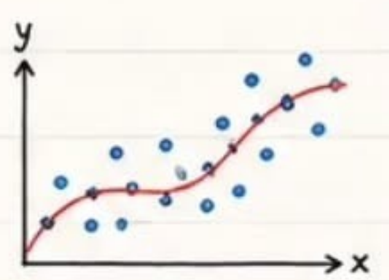
Uses multiple features to predict target.

Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \varepsilon$$

Use Case:
House price prediction, salary prediction, etc.

Polynomial Regression



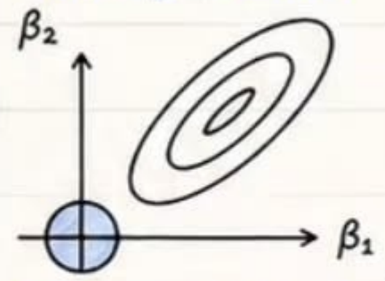
Models non-linear relationship using polynomial features.

Equation (degree d):

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_d x^d + \varepsilon$$

Use Case:
Demand forecasting, growth modeling, etc.

Ridge Regression (L2 Regularization)



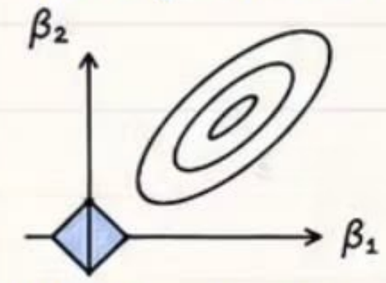
Adds L2 penalty to reduce overfitting by shrinking coefficients.

Cost Function:

$$J = MSE + \lambda \sum_{i=1}^{n} \beta_i^2$$

Use Case:
When features are many or multicollinear.

Lasso Regression (L1 Regularization)



Adds L1 penalty which can make some coefficients exactly zero (feature selection).

Cost Function:

$$J = MSE + \lambda \sum_{i=1}^{n} |\beta_i|$$

Use Case:
Feature selection, sparse models, high-dimensional data.

② Common Assumptions (Linear Regression)

- ✓ Linearity: Relationship between X and y is linear.
- ✓ Independence: Observations are independent.
- ✓ Homoscedasticity: Constant variance of errors.
- ✓ Normality: Errors are normally distributed.
- ✓ No Multicollinearity: Features are not highly correlated.

③ Summary Comparison

Algorithm	Model Type	Handles Non-linearity	Regularization	Feature Selection	Overfitting Control
Linear Regression	Linear	✗	None	✗	Low
Multiple Regression	Linear	✗	None	✗	Low
Polynomial Regression	Non-linear	✓	None	✗	Medium
Ridge Regression	Linear	✗	L2	✗	High
Lasso Regression	Linear	✗	L1	✓	High

④ Python Example (Linear Regression)

```python
from sklearn.linear_model import LinearRegression
import pandas as pd

data = pd.read_csv('house_prices.csv')
X = data[['area', 'bedrooms']]  # features
y = data['price']               # target

model = LinearRegression()
model.fit(X, y)
pred = model.predict([[1200, 3]])
print(pred)
```

Note:
Choose the right algorithm based on data pattern and problem type.

Interview Tip

- ✗ Linear regression is simple but assumptions must be checked.
- ✗ Ridge helps with multicollinearity.
- ✗ Lasso helps in feature selection.

⑤ Real-World Applications

- House Price Prediction
- Sales & Revenue Forecasting
- Demand Forecasting
- Salary Prediction
- Temperature Prediction
- Health Risk Score Prediction

⚠ Common Mistakes

- ✗ Ignoring outliers and influential points.
- ✗ Using regression for classification problems.
- ✗ Not scaling features before regularized models.
- ✗ Overfitting with high-degree polynomials.

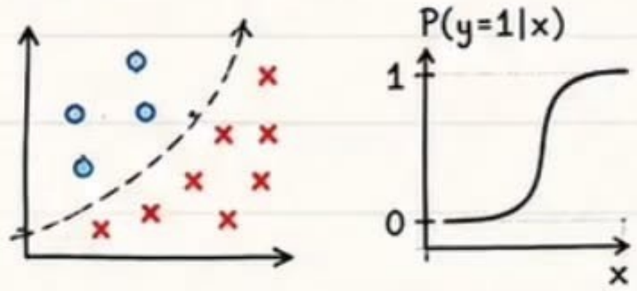
SRIPADH K

Classification Algorithms

Classification predicts a discrete class label for input data.
Algorithms learn a decision boundary that separates classes.

① Popular Classification Algorithms

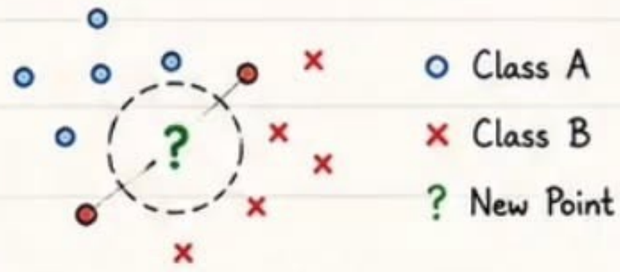
① Logistic Regression



Linear model used for binary classification.

Outputs probability using sigmoid function.

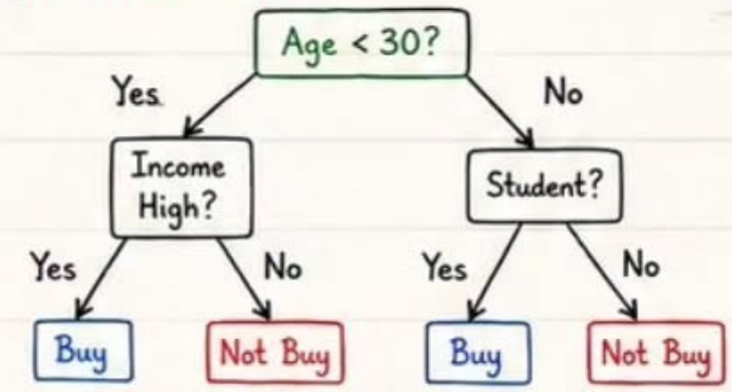
② K-Nearest Neighbors (KNN)



Non-parametric algorithm.

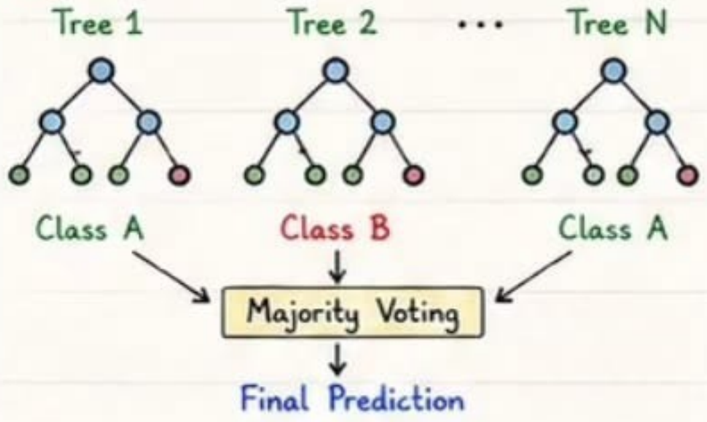
Class of a point is decided by the majority class of its K neighbors.

③ Decision Tree



Tree-like model that splits data using feature conditions.

④ Random Forest



Ensemble of decision trees.
Reduces overfitting and improves accuracy.

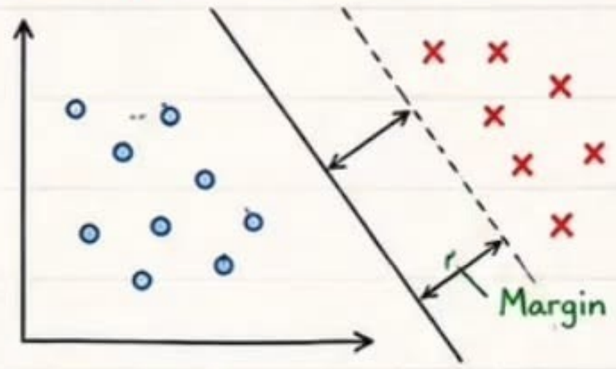
⑤ Naive Bayes

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

Feature	Class	
	A	B
X_1	$P(X_1\|A)$	$P(X_1\|B)$
X_2	$P(X_2\|A)$	$P(X_2\|B)$
...		

Probabilistic algorithm based on Bayes' Theorem with independence assumption.
Works well for text, spam, and NLP tasks.

⑥ Support Vector Machine (SVM)



Finds the optimal hyperplane that maximizes the margin between classes.
Effective in high-dimensional spaces.

② Algorithm Comparison

Algorithm	Model Type	Decision Boundary	Works Well When	Pros	Cons
Logistic Regression	Linear	Linear	Linear relation, small/medium data	✓ Simple, Fast, Interpretable	✗ Underfits complex data
KNN	Non-Parametric	Non-Linear	Small data, non-linear boundary	✓ Simple, No training phase	✗ Slow prediction, Sensitive to K
Decision Tree	Non-Linear	Non-Linear	Mixed data types, easy to interpret	✓ Interpretable, Handles non-linear	✗ Overfitting, High variance
Random Forest	Ensemble	Non-Linear	Large datasets, high variance	✓ High accuracy, Robust	✗ Less interpretable
Naive Bayes	Probabilistic	Linear / Non-Linear	Text data, high dimensional sparse	✓ Fast, Works with small data	✗ Independence assumption
SVM	Linear / Non-Linear	Linear / Non-Linear	High dimensional, clear margin	✓ Effective in high dimensions	✗ Slow on large data

Key Takeaways

- ✓ Choose algorithm based on data, problem type and business requirement.
- ✓ No single algorithm is best for all problems.
- ✓ Try multiple models and compare performance.
- ✓ Validate using proper metrics and test data.

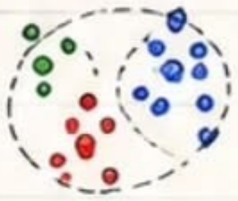
Interview Tip

- ☆ Know the intuition, strengths, limitations and use cases.
- ☆ Be ready to explain decision boundary and overfitting control techniques.
- ☆ Understand bias-variance trade-off.

Common Mistakes

- ✗ Using accuracy alone for evaluation.
- ✗ Ignoring feature scaling for KNN & SVM.
- ✗ Not handling class imbalance.
- ✗ Skipping cross-validation.
- ✗ Overfitting with complex models.

Real-World Applications: Spam Detection | Fraud Detection | Customer Churn Prediction | Medical Diagnosis | Image Classification



Unsupervised Learning

Unsupervised Learning finds hidden patterns or structure in data without using any labels (target).

① Clustering Concept

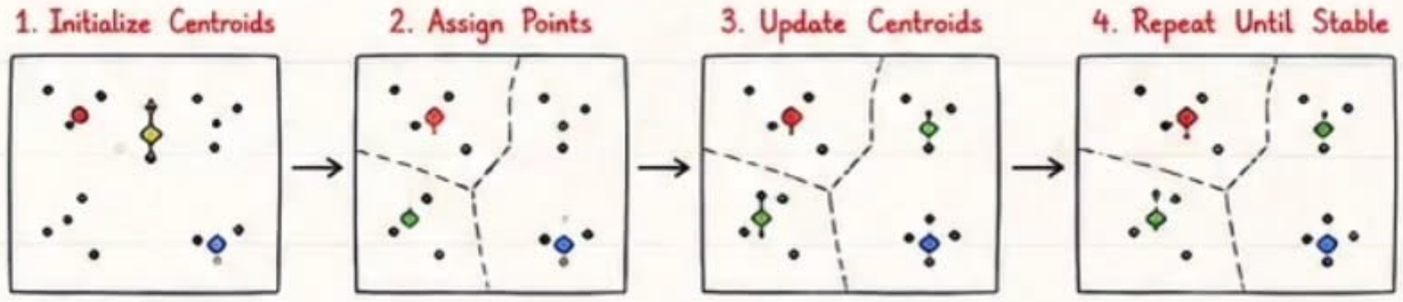
- Group similar data points together based on their features.
- Points in the same cluster are more similar to each other.
- No target variable is needed.

Use Cases:

- ✓ Customer Segmentation
- ✓ Market Basket Analysis
- ✓ Document Grouping
- ✓ Image Segmentation

② K-Means Clustering

- Partitions data into K clusters.
- Assigns each point to the nearest centroid.
- Updates centroid as mean of points.
- Iterates until convergence.

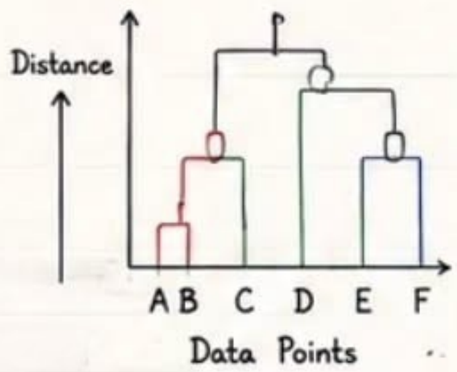


Minimize $J = \sum_{i=1}^{K} \sum_{x \in C_i} \| x - \mu_i \|^2$

Where, μ_i = centroid of cluster i
C_i = points in cluster i

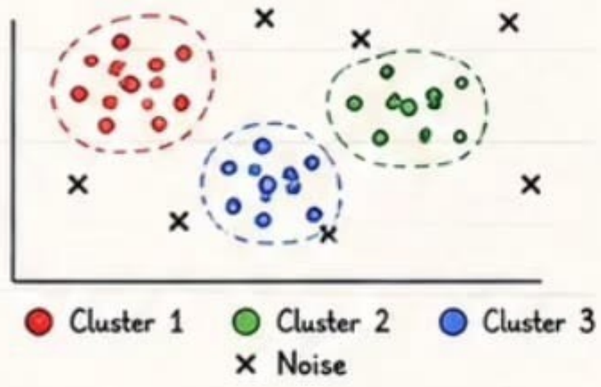
③ Hierarchical Clustering

- Builds a hierarchy of clusters.
- Two types:
  - Agglomerative (Bottom-Up)
  - Divisive (Top-Down)
- Visualized using Dendrogram.



④ DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- Groups points that are closely packed together.
- Identifies noise / outliers as points in low-density regions.
- Parameters:
  eps (neighborhood radius)
  min_samples (minimum points)



⑤ Algorithm Comparison

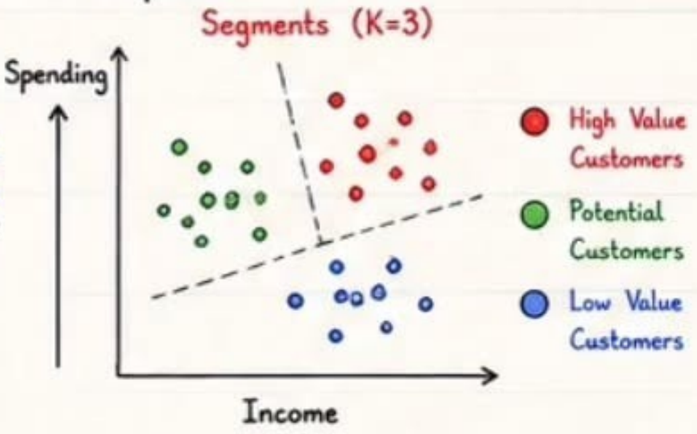
Algorithm	Approach	Clusters Shape	Need K?	Noise Handling	Scalability	Best Use
K-Means	Centroid Based	Spherical (Convex)	Yes	No	High	Large datasets, well-separated clusters
Hierarchical	Tree Based	Any Shape	No	No	Low	Small datasets, exploratory analysis
DBSCAN	Density Based	Arbitrary Shape	No	Yes	Medium	Clusters with noise, outlier detection

⑥ Customer Segmentation Example

Customer Data

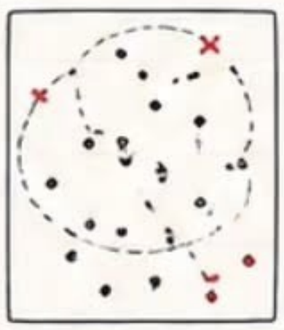
Income	Spending	Age
40k	2k	25
60k	5k	40
30k	1k	22
80k	6k	50
...	...	...

K-Means Clustering →



⑦ Anomaly Detection (Unsupervised)

- Finds rare patterns that don't conform to normal behavior.
- Methods: Isolation Forest, One-Class SVM, Autoencoders, DBSCAN (noise).
- Use Cases: Fraud Detection, Fault Detection, Network Intrusion, Medical Diagnosis.



Interview Tip

- ☆ K-Means is simple but assumes spherical clusters.
- ☆ DBSCAN is best for noise and arbitrary shapes.
- ☆ Hierarchical gives hierarchy (dendrogram).

Best Practices

- ✓ Scale features before clustering.
- ✓ Choose right algorithm based on data and problem.
- ✓ Use domain knowledge to interpret clusters.

Common Mistakes

- ✗ Using K-Means when K is unknown.
- ✗ Ignoring feature scaling.
- ✗ Assuming all clusters are spherical.
- ✗ Not handling outliers or noise.
- ✗ Over-interpreting clusters without domain knowledge.

Feature Engineering

Feature Engineering is the art of creating better features from raw data to improve model performance, reduce overfitting and capture patterns.

1 Why is Feature Engineering Important?

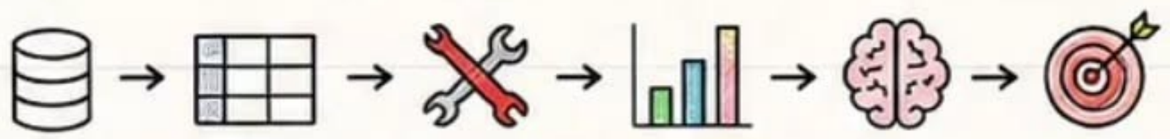
- ✓ Improves model accuracy.
- ✓ Reduces training time.
- ✓ Helps models generalize better.
- ✓ Turns domain knowledge into useful signals.
- ✓ Sometimes more important than the algorithm!

Key Takeaway

Good features >> Good Model

- ✓ Better representation of data
- ✓ Stronger patterns
- ✓ Improved generalization

2 Feature Engineering Workflow



3 Types of Feature Engineering

1. Feature Creation

Create new useful features from existing ones.

Example:

Total = Price x Qty

Discount(%) = Discount / Price

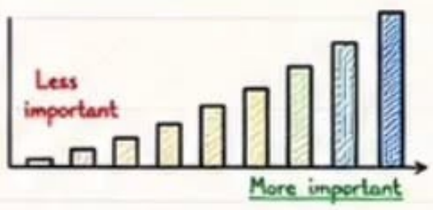
Price	Qty	Total
100	3	300
200	2	400

2. Feature Selection

Select the most important features and remove noise.

Methods:

- Correlation
- Variance Threshold
- Chi-Square
- Mutual Information
- Model Based (RF, L1)



3. Feature Transformation

Transform features to make them more suitable for models.

Examples:

- Log Transformation
- Power Transformation
- Date/Time Features
- Aggregations

4. Encoding Categorical

Convert categorical variables into numeric format.

Examples:

- Label Encoding
- One-Hot Encoding
- Ordinal Encoding

Color
Red
Blue
Green

→

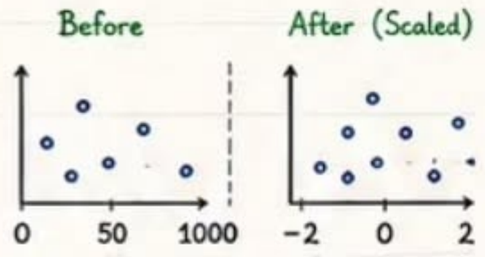
Red	Blue	Green
1	0	0
0	1	0
0	0	1

5. Feature Scaling

Scale numeric features to similar range.

Methods:

- Normalization (0-1)
- Standardization (Z-score)
- Robust Scaling



4 Binning (Discretization)

Convert continuous data into bins. Helps in handling non-linear patterns and outliers.

Example (Age):

Age	Bin
0 - 18	Young
18 - 35	Adult
35 - 60	Middle Aged
60+	Senior

5 Interaction Features

Combine two or more features to capture hidden patterns.

Example:

Income	Spending	Income x Spending
50000	20000	1,000,000
60000	25000	1,500,000
40000	15000	600,000

Helps models understand relationships between features.

6 Use Domain Knowledge

Domain knowledge helps in:

- ✓ Identifying important features.
- ✓ Creating meaningful features.
- ✓ Removing irrelevant information.
- ✓ Interpreting model behavior.

Example (E-commerce):

- Combine product category & price.
- Day of week, holiday flag, season.
- User's past behavior & recency.

7 Python Example (Feature Engineering)

```
import pandas as pd
df = pd.read_csv('sales.csv')
# Feature Creation
df['Total'] = df['Price'] * df['Quantity']
# Binning
df['Age_Group'] = pd.cut(df['Age'],
                         bins=[0,18,35,60,100],
                labels=['Young','Adult','Middle_Aged','Senior'])
# One-Hot Encoding
df = pd.get_dummies(df, columns=['City'], drop_first=True)
```

Interview Tip

- ☆ Explain WHY a feature is useful.
- ☆ Talk about impact on model.
- ☆ Mention techniques you used.
- ☆ Always validate with experiments.

Best Practices

- ✓ Understand data before creating features.
- ✓ Avoid data leakage while creating features.
- ✓ Keep features simple yet meaningful.
- ✓ Check multicollinearity.
- ✓ Validate feature importance.

Common Mistakes

- ✗ Creating features using test data.
- ✗ Over-engineering (too many features).
- ✗ Ignoring feature scaling/encoding.
- ✗ Not checking multicollinearity.
- ✗ Not using domain knowledge.

Model Evaluation Metrics

Evaluation metrics help us measure how well our model performs on unseen data and make the right decisions.

① Confusion Matrix (For Classification)

Predicted Values \ Actual Values	Positive (1)	Negative (0)
Positive (1)	TP (True Positive)	FP (False Positive)
Negative (0)	FN (False Negative)	TN (True Negative)

- **TP**: Predicted positive and actually positive.
- **FP**: Predicted positive but actually negative.
- **FN**: Predicted negative but actually positive.
- **TN**: Predicted negative and actually negative.

② Classification Metrics

- **Accuracy** = $\frac{TP + TN}{TP + TN + FP + FN}$ — Overall correctness.
- **Precision** = $\frac{TP}{TP + FP}$ — How many predicted positives are correct.
- **Recall (Sensitivity)** = $\frac{TP}{TP + FN}$ — How many actual positives are captured.
- **F1 Score** = $2 \times \frac{Precision \times Recall}{Precision + Recall}$ — Balance between Precision & Recall.

③ Example Confusion Matrix

Predicted \ Actual	Yes (1)	No (0)
Yes (1)	50	10
No (0)	5	35

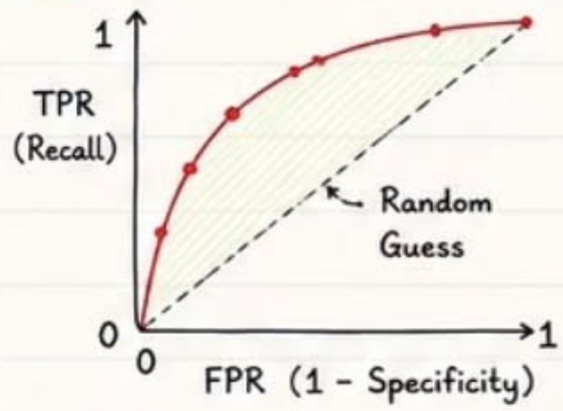
From the matrix:

- TP = 50
- FP = 10
- FN = 5
- TN = 35

Calculated Metrics

- Accuracy = 85%
- Precision = 83.33%
- Recall = 90.91%
- F1 Score = 86.96%

④ ROC Curve & ROC-AUC



- ROC curve shows trade-off between TPR and FPR.
- AUC (Area Under Curve) measures separability.
- AUC = 1 → Perfect model
- AUC = 0.5 → Random guess
- Higher AUC is better.

⑤ Regression Metrics

- **MAE (Mean Absolute Error)** = $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$ — Average absolute prediction error.
- **MSE (Mean Squared Error)** = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ — Penalizes large errors more.
- **RMSE (Root Mean Squared Error)** = $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$ — Same unit as target, easier to interpret.
- **R^2 (R-Squared)** = $1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$ — Proportion of variance explained by model.

Where: y_i = actual value, $\hat{y}_i$ = predicted value, $\bar{y}$ = mean of actual values, n = number of samples

⑥ Metric Comparison

Metric	Best Used For	What it Measures	Range	Better
Accuracy	Balanced Classes	Overall correctness	0 to 1	Higher
Precision	High FP cost	Correct positive predictions	0 to 1	Higher
Recall	High FN cost	Capture actual positives	0 to 1	Higher
F1 Score	Imbalanced Classes	Balance of P & R	0 to 1	Higher
ROC-AUC	Probabilistic Models	Separability	0 to 1	Higher
MAE	Regression	Average absolute error	0 to ∞	Lower
MSE	Regression	Average squared error	0 to ∞	Lower
RMSE	Regression	Square root of MSE	0 to ∞	Lower
R^2 Score	Regression	Variance explained	-∞ to 1	Higher

⑦ Choosing the Right Metric

- ✓ Understand the business goal and cost of errors.
- ✓ Check class distribution (balanced vs imbalanced).
- ✓ Use Precision/Recall/F1 for imbalanced data.
- ✓ Use ROC-AUC to compare probabilistic models.
- ✓ Use MAE/RMSE for regression problems.
- ✓ Always validate on test data, not training data.

☆ Remember

No single metric is best for all problems. Pick the metric that aligns with your goal!

⑧ Real-World Examples

- Fraud Detection → Recall, ROC-AUC
- Disease Prediction → Recall, F1 Score
- Recommendation → Precision, AUC
- House Price Prediction → RMSE, MAE, R^2

⑨ Tips & Best Practices

- ✓ Evaluate on unseen test data.
- ✓ Compare multiple metrics.
- ✓ Use cross-validation for stability.
- ✓ Visualize (ROC curve, error plots).
- ✓ Understand the trade-offs.
- ✓ Never optimize for Accuracy alone!

⚠ Common Mistakes

- ✗ Using Accuracy on imbalanced data.
- ✗ Ignoring business cost of errors.
- ✗ Data leakage during evaluation.
- ✗ Not setting proper baseline.
- ✗ Overritting while tuning for metric.

Cross-Validation & Hyperparameter Tuning

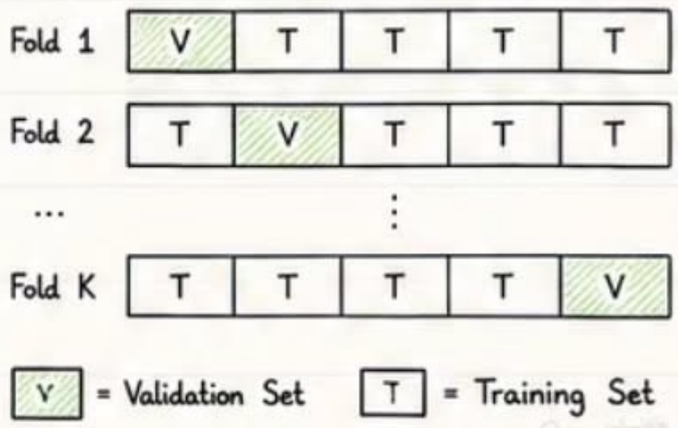
Cross-Validation helps us evaluate model performance reliably.
Hyperparameter Tuning finds the best settings for the model.

① Parameter vs Hyperparameter

Parameter	Hyperparameter
Values learned by the model from training data.	Values set by the user before training.
Change during training process.	Control the learning process.
Example: weight (w), bias (b) in Linear Regression.	Example: learning_rate, n_estimators, max_depth, C, k, alpha.

② K-Fold Cross-Validation

Split data into K parts. Use K-1 parts for training and 1 part for validation.
Repeat K times and average the score.



Final Score = Average of K validation scores

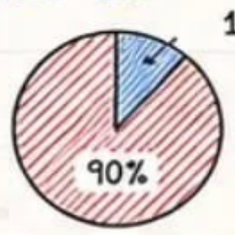
③ Stratified K-Fold

Maintains the same class distribution in each fold. Important for imbalanced classification problems.

Example (Imbalanced Data)

Class 1: 90%

Class 0: 10%



Each fold will have similar 90:10 ratio.

When to Use?

- ✓ Classification with imbalanced classes
- ✓ When target distribution matters

④ Cross-Validation Strategies

Strategy	Uses	Pros	Cons
K-Fold CV	General problems	Simple, reduces variance	May not preserve class ratio
Stratified K-Fold CV	Imbalanced classification	Keeps class distribution	Slightly more complex
Leave-One-Out (LOOCV)	Small datasets	Low bias	High variance, computationally expensive
Time Series Split	Time series data	Respects time order	Less data for training

⑤ Hyperparameter Tuning Methods

Grid Search

Try all possible combinations of hyperparameters.
Exhaustive search.

Example:

```
params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7]
}
```

Total models = 3 x 3 = (9)

Random Search

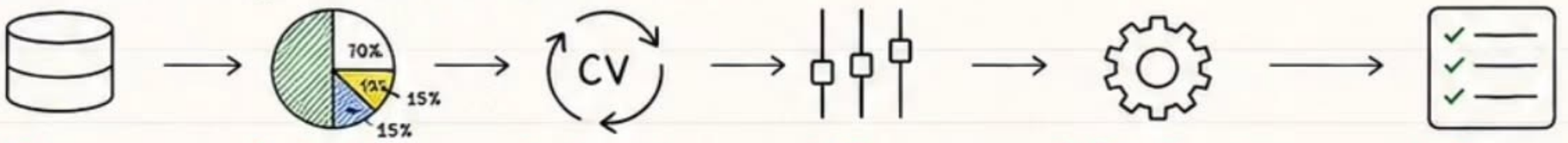
Try random combinations from parameter space.
More efficient for large spaces.

Example:

```
params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7]
}
```

Total models = (5) (random picks)

⑥ Validation Strategy in Model Development



1. Prepare Dataset
2. Split Data
   - 70% Training
   - 15% Validation
   - 15% Test
3. Cross-Validate on Training Set
   Use K-Fold (or Stratified K-Fold)
4. Tune Hyperparameters
   Grid Search / Random Search
5. Train Final Model on Train + Val
   Use best hyperparameters on full training data
6. Evaluate on Test Set
   Get unbiased performance

⑦ Python Example (GridSearchCV)

```
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5].
}

cv = KFold(n_splits=5, shuffle=True, random_state=42)
model = RandomForestClassifier(random_state=42)
grid = GridSearchCV(model, param_grid, cv=cv, scoring='f1')
grid.fit(X_train, y_train)

print("Best Params:", grid.best_params_)
print("Best CV Score:", grid.best_score_)
```

Interview Facts

- ☆ Cross-validation reduces variance and gives stable estimate.
- ☆ Stratified K-Fold is must for imbalanced classification.
- ☆ Random Search is preferred over Grid Search for large hyperparameter space.

Best Practices

- ✓ Always use CV for robust evaluation.
- ✓ Avoid data leakage during CV.
- ✓ Use appropriate scoring metric.
- ✓ Tune one model at a time.
- ✓ Validate final model on unseen test data only.

Common Mistakes

- ✗ Tuning on test data.
- ✗ Not shuffling data before K-Fold (when applicable).
- ✗ Ignoring class imbalance in CV.
- ✗ Over-tuning leading to overfitting.
- ✗ Using accuracy for all problems.

SRIPADH K

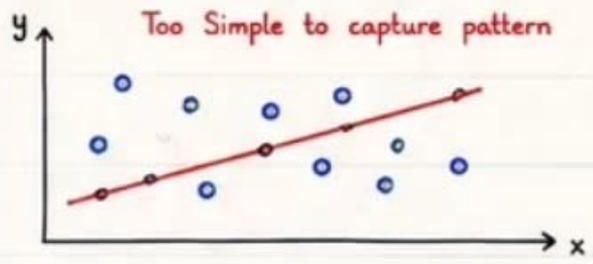
Overfitting, Underfitting & Bias-Variance

A good model should generalize well to unseen data.
It should have the right balance between Bias and Variance.

1 Key Concepts

- **Bias**: Error due to overly simple assumptions in the model.
- **Variance**: Error due to sensitivity to small fluctuations in training data.
- **Model Complexity** controls the bias-variance trade-off.
- **Goal**: Minimize total error to achieve good generalization.

2 Underfitting (High Bias)

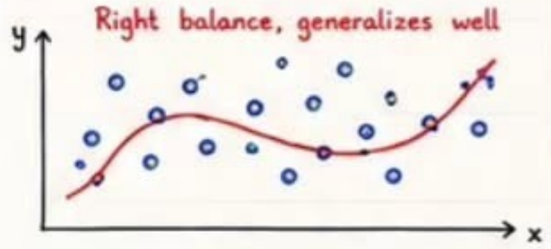


- High bias, Low variance.
- Fails to learn from data.
- High training error & high test error.
- Example: Linear model for complex non-linear data.

3 Overfitting (High Variance)

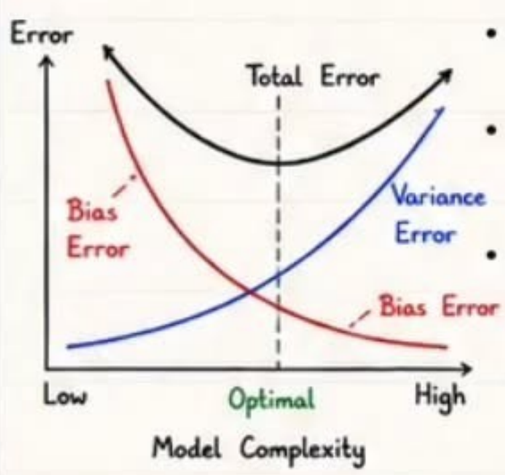
- Low bias, High variance.
- Fits noise in training data.
- Low training error & high test error.
- Example: High degree polynomial, deep tree with many leaves.

4 Good Fit (Balanced)



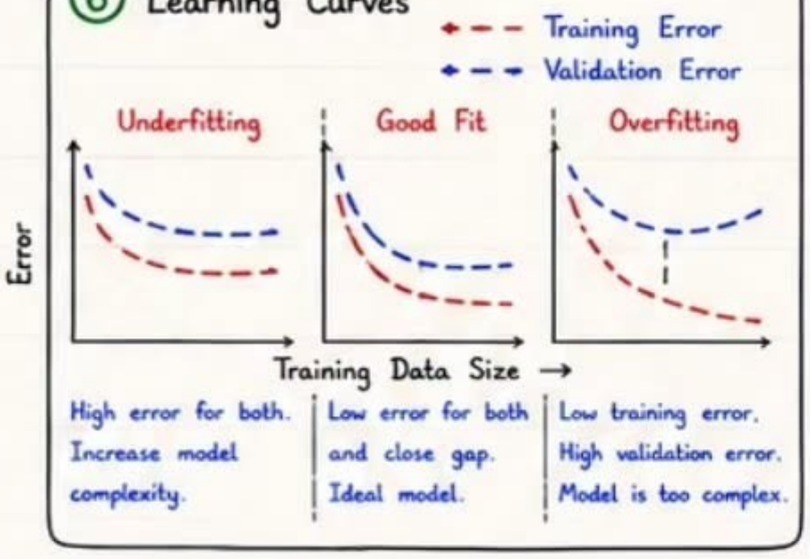
- Low bias, Low variance (balanced).
- Captures underlying pattern.
- Low training & low test error.
- Best generalization to unseen data.

5 Bias-Variance Trade-off



- Low complexity → High bias, Low variance.
- High complexity → Low bias, High variance.
- Choose the optimal complexity where total error is minimum.

6 Learning Curves



Underfitting	Good Fit	Overfitting
High error for both. Increase model complexity.	Low error for both and close gap. Ideal model.	Low training error. High validation error. Model is too complex.

7 Causes & Solutions

Issue	Cause	Symptoms	Solutions
Overfitting (High Variance)	• Too complex model • Too many features • Small dataset / Noise	• Low training error • High validation error • High variance	✓ Use regularization (L1 / L2) ✓ Simplify model / Reduce features ✓ More training data / Drop noise ✓ Cross-validation / Early stopping
Underfitting (High Bias)	• Too simple model • Missing important features • Strong assumptions	• High training error • High validation error • High bias	✓ Increase model complexity ✓ Add meaningful features ✓ Reduce strong assumptions ✓ Train longer / Better features
Good Fit (Balanced)	• Right complexity • Good features • Enough data	• Low training error • Low validation error • Good generalization	✓ Maintain current approach ✓ Monitor with validation ✓ Avoid unnecessary complexity

8 Regularization

Adds penalty to reduce model complexity.

- L2 (Ridge):

$$J = \text{MSE} + \lambda \sum_{i=1}^{n} w_i^2$$

- L1 (Lasso):

$$J = \text{MSE} + \lambda \sum_{i=1}^{n} |w_i|$$

λ → regularization strength
(higher λ = stronger penalty)

Practical Tips

- ✓ Always visualize learning curves.
- ✓ Use cross-validation.
- ✓ Try simpler model first, then increase complexity.
- ✓ Validate on unseen test data.
- ✓ Feature engineering > complex models sometimes.

Interview Facts

- Bias: error from wrong assumptions.
- Variance: error from sensitivity to data.
- No free lunch theorem – no single model is best for all problems.
- Aim for balanced bias-variance.

Common Mistakes

- ✗ Using very complex model on small data.
- ✗ Ignoring validation set.
- ✗ Not checking learning curves.
- ✗ Over-regularizing (λ too high).
- ✗ Using training accuracy as final metric.

Key Takeaway

Good models are not too simple and not too complex.
Balance Bias & Variance for best generalization!

Understand Data → Choose Right Complexity → Validate & Tune → Good Generalization

Advanced ML & Ensemble Learning

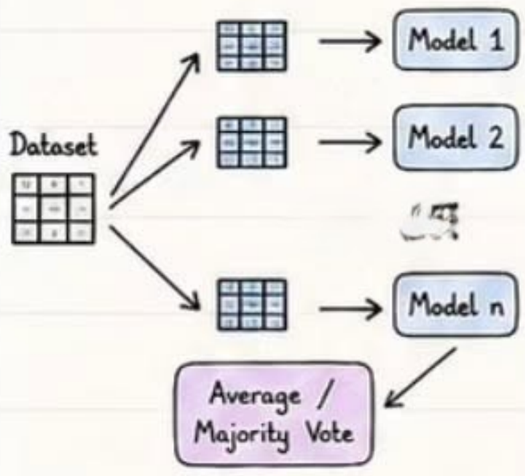
Ensemble Learning combines multiple models to get better accuracy, stability and generalization than a single model.

① Ensemble Learning Paradigms

A. Bagging

Reduce variance by training models in parallel on different samples.

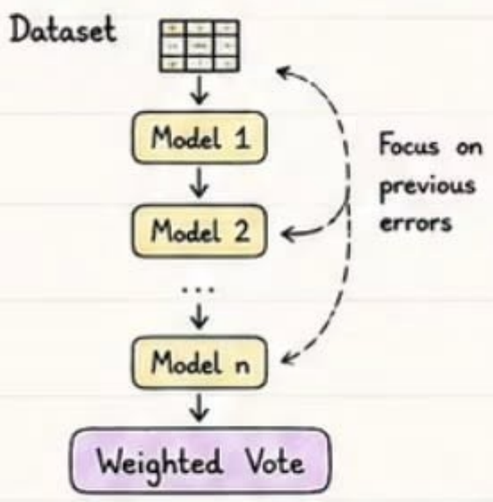
Example: Random Forest



B. Boosting

Reduce bias (and some variance) by training models sequentially. Each model focuses on correcting previous errors.

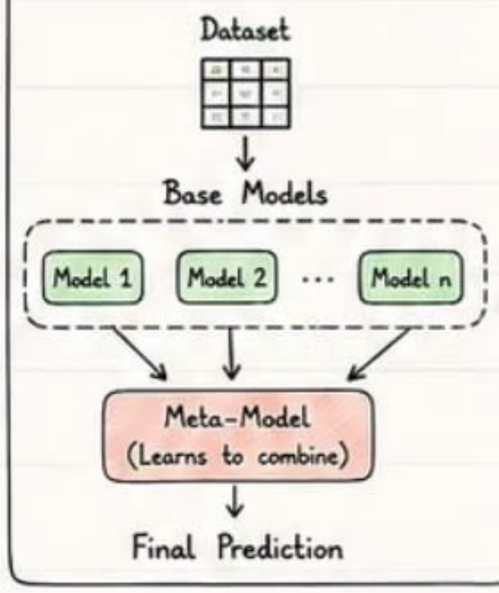
Example: AdaBoost, GBM, XGBoost



C. Stacking

Combine different models using another model (meta-learner).

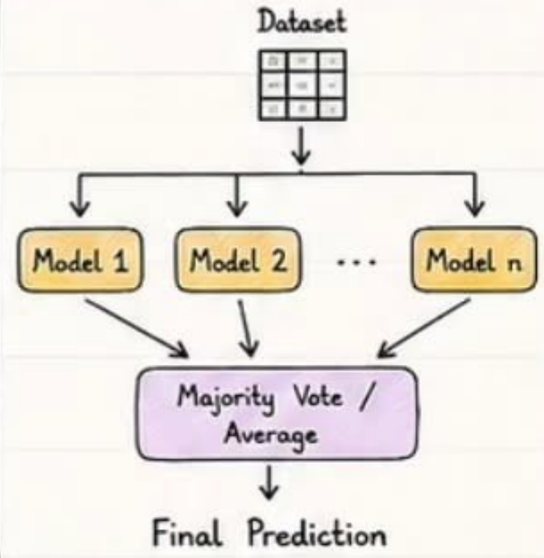
Example: Stacking Classifier



D. Voting

Combine predictions by simple majority or averaging.

Example: Hard & Soft Voting



② Popular Ensemble Algorithms

Algorithm	Type	Key Idea	Strengths	Use Cases
Random Forest	Bagging	Many Decision Trees on bootstrap samples + feature randomness.	✓ Reduces overfitting ✓ Works well on tabular data & large datasets	Fraud detection, churn prediction, credit risk
Gradient Boosting	Boosting	Sequential trees; each tree fixes previous errors.	✓ High accuracy ✓ Handles complex patterns	Marketing response, forecasting, ranking
XGBoost	Boosting	Optimized GBM with regularization, parallel processing.	✓ Fast, scalable, handles missing values ✓ Prevent overfitting	Kaggle competitions, structured data problems
LightGBM	Boosting	Leaf-wise tree growth, histogram based.	✓ Very fast & memory efficient ✓ Better for large datasets	CTR prediction, ranking, large scale applications
CatBoost	Boosting	Handles categorical features natively with ordered boosting.	✓ Excellent with categorical data ✓ Less preprocessing	Finance, ecommerce, customer behavior

③ When Ensembles Outperform?

- ✓ Complex, non-linear relationships
- ✓ High dimensional data
- ✓ Noisy or imbalanced datasets
- ✓ Need high predictive accuracy
- ✓ When individual models are high variance (like Decision Trees)
- ✓ Real-world problems with diverse patterns

☆ Key Takeaway

Ensembles combine the wisdom of many weak/strong models to make one strong, robust and accurate model.

④ How They Complement Bias-Variance Trade-off

⑤ Python Example (Random Forest)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
X, y = load_iris(return_X,y=True)
model = RandomForestClassifier(
   n_estimators=100,                    # Bagging ensemble
   max_depth=None,                      # 100 decision trees
   random_state=42)
)
model.fit(X, y)                         # Train
print("Accuracy:", model.score(X, y))   # Avaluate on training data (demo)
```

⑥ Quick Comparison

Aspect	Bagging (RF)	Boosting (GBM/XGB/LGBM/CatBoost)	Stacking	Voting
Training	Parallel	Sequential	Two-level	Parallel
Reduces	Variance	Bias (and Variance)	Both	Variance
Complexity	Medium	High	High	Low
Speed	Fast	Slower	Slower	Fast
Use When	Overfitting	Underfitting	Best generalization	Simple & effective

Interview Tip

- ☆ Bagging = Parallel + Variance reduction
- ☆ Boosting = Sequential + Bias reduction
- ☆ Stacking = Heterogeneous models
- ☆ Ensembles almost always beat single models on real-world data.

Best Practices

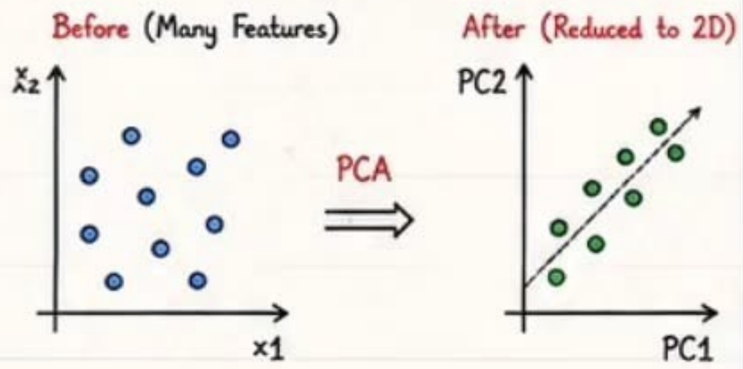
- ✓ Tune hyperparameters carefully
- ✓ Use cross-validation
- ✓ Avoid data leakage
- ✓ Analyze feature importance
- ✓ Use early stopping in boosting

Dimensionality Reduction & Model Interpretability

Dimensionality Reduction simplifies data by keeping important information.
Model Interpretability helps us understand WHY a model makes a prediction.

1 Why Reduce Dimensions?

- ✓ High-dimensional data increases complexity and training time.
- ✓ Many features may be redundant or noisy.
- ✓ Helps in visualization (2D/3D).
- ✓ Improves model performance and generalization.



2 PCA (Principal Component Analysis)

- ✓ Transforms original correlated features into new uncorrelated components (PCs).
- ✓ PC1 captures maximum variance, PC2 next, and so on.
- ✓ Helps reduce dimensions while preserving information.

How PCA Works?

1. Standardize data
2. Compute covariance matrix
3. Compute eigenvalues & eigenvectors
4. Select top 'k' PCs
5. Transform the data

3 Feature Selection vs Dimensionality Reduction

Aspect	Feature Selection	Dimensionality Reduction
Goal	Select a subset of original features	Create new features (combinations) of originals
Interpretability	High	Lower (PCs are combinations)
Information Loss	Minimal	Some information may be lost
Examples	Filter (Corr, Chi²), Wrapper (RFE), Embedded (Lasso)	PCA, t-SNE, UMAP, Autoencoders
Best For	When we want simpler & interpretable models	When features are highly correlated or very high-dimensional

4 Common Dimensionality Reduction Techniques

Technique	Type	Use Case
PCA	Linear	Generad purpose, reduce correlated features
t-SNE	Non-linear	Visualization of high-dimensional data
UMAP	Non-linear	Visualization + faster than t-SNE
Autoencoders	Non-linear	Learn compressed representation (Deep Learning)

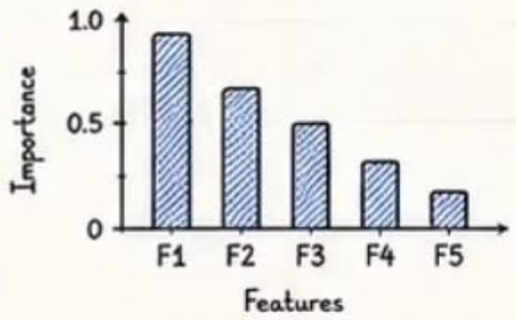
PCA Explained

PCA finds new axes (PCs) that capture maximum variance.
We keep top 'k' PCs that explain most of the variance.

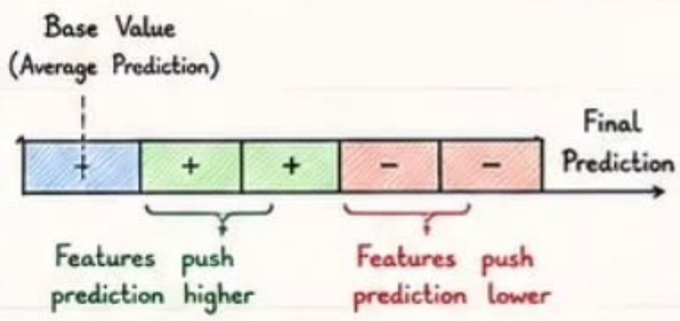
$$\text{Explained Variance Ratio} = \frac{\text{Variance captured by selected PCs}}{\text{Total Variance in data}}$$

5 Feature Importance

- ✓ Indicates how much each feature contributes to the prediction.
- ✓ Helps in model understanding, debugging and feature selection.
- ✓ Tree-based models provide built-in importance scores.

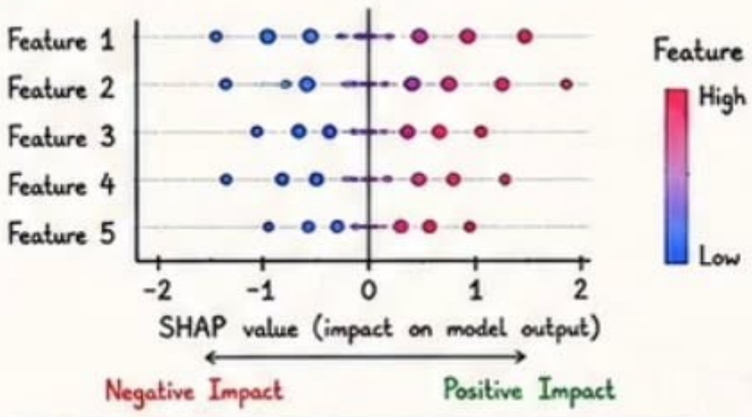


6 SHAP (SHapley Additive exPlanations)



- ✓ Based on game theory (Shapley values).
- ✓ Explains individual prediction.
- ✓ Shows how each feature contributes positively (+) or negatively (−).
- ✓ Model-agnostic (works with any model).
- ✓ Global (summary) & Local (per prediction) explanations.

7 Example: SHAP Summary Plot (Concept)



- Each dot = one sample.
- Color shows feature value (High → Red, Low → Blue).
- Right side → increases prediction.
- Left side → decreases prediction.

8 When to Use What?

Scenario	Recommended Approach
Many correlated features	PCA
Need interpretable features	Feature Selection
High-dimensional text/image data	PCA / Autoencoders
Need visualization (2D/3D)	t-SNE / UMAP
Explain individual prediction	SHAP
Understand model globally	Feature Importance + SHAP Summary

Interview Facts

- ☆ PCA is unsupervised.
- ☆ Feature selection keeps original features.
- ☆ SHAP values sum up to (prediction − base value).

Best Practices

- ☆ Always scale data before PCA.
- ☆ Avoid too many features → risk of overfitting.
- ☆ Use interpretability to build trust in ML models.

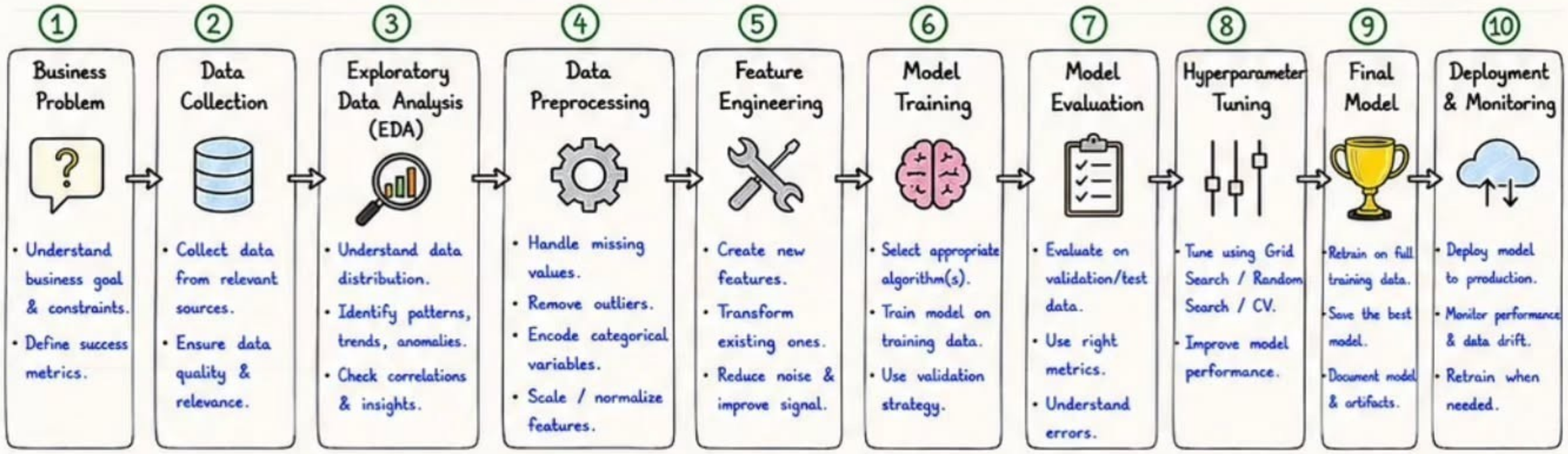
Common Mistakes

- ✗ Applying PCA without scaling.
- ✗ Using too many PCs → noise retained.
- ✗ Ignoring feature importance in complex models.

SRIPADH K

Complete End-to-End ML Project

A successful ML project follows a structured workflow from understanding the business problem to delivering a production-ready model that creates value.



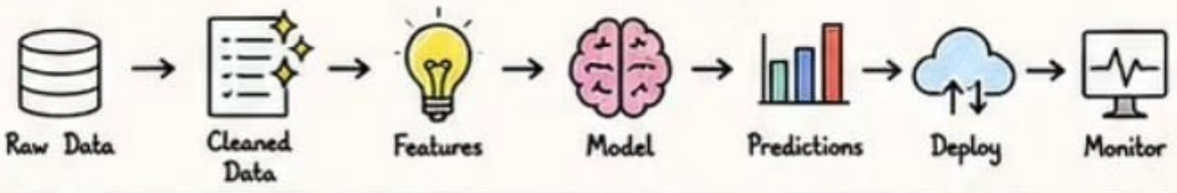
Key Activities in Detail

① Business Problem : Define objective, target metric, scope and constraints.
② Data Collection : Gather data from DB, APIs, files, logs, surveys, etc.
③ EDA : Visualize data, analyze distributions, detect missing values, correlations.
④ Preprocessing : Clean data, encode, scale, handle imbalance if any.
⑤ Feature Engineering : Add interaction, polynomial, domain-driven features.
⑥ Model Training : Try multiple algorithms, build baseline model.
⑦ Evaluation : Compare models using appropriate metrics & CV.
⑧ Tuning : Optimize hyperparameters for best generalization.
⑨ Final Model : Train final model, save pipeline & model artifacts.
⑩ Deployment & Monitoring : Serve model, monitor metrics & retrain.

Typical Data Split

Set	Purpose	Typical Ratio
Training Set	Learn patterns from data	60% – 80%
Validation Set	Tune hyperparameters & model selection	10% – 20%
Test Set	Final unbiased evaluation	10% – 20%

Sample End-to-End ML Pipeline



Common Evaluation Metrics

Classification
- Accuracy, Precision, Recall
- F1 Score
- ROC-AUC
- Log Loss

Regression
- MAE, MSE, RMSE
- R^2 Score
- MAPE

Python Sklearn Pipeline Example

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=200, random_state=42))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print('Test Accuracy :', pipeline.score(X_test, y_test))
```

Why Follow This Workflow?

✓ Ensures high quality & reliable models.
✓ Improves accuracy & generalization.
✓ Easy to reproduce and maintain.
✓ Helps in scaling to production.

Key Takeaway

A well-structured ML pipeline + good data + right evaluation = Strong & trustworthy model that delivers real business value.

Best Practices

✓ Always start with clear business goal.
✓ Spend more time on data understanding.
✓ Prevent data leakage at every step.
✓ Track experiments and results.
✓ Document everything for reproducibility.

Interview Facts

☆ 80% of ML time is spent on data preparation.
☆ No perfect model – just the right model for the problem.
☆ Good evaluation strategy > complex model.
☆ Always validate on unseen test data.

Common Mistakes

✗ Not understanding the business problem.
✗ Training and testing on the same data.
✗ Ignoring data quality & missing values.
✗ Overfitting by not using validation.
✗ Not monitoring the model after deployment.

SRIPADH K

17

ML Deployment & MLOps

MLOps is the practice of deploying, monitoring and maintaining ML models in production reliably and at scale.

① Model Serialization

- Serialize (save) the trained model and related objects so they can be used later.

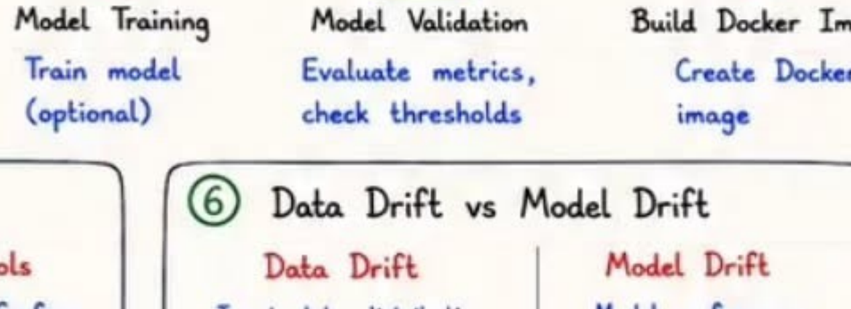
Common Formats:

- joblib / pickle → Python objects
- ONNX → Cross platform
- SavedModel → TensorFlow
- TorchScript → PyTorch

```
# Using joblib
import joblib
joblib.dump(model, 'model.pkl')
loaded_model = joblib.load('model.pkl')
```

② API Deployment

Expose model as an API so applications can send data and get predictions.



FastAPI (Modern & Fast)

```
from fastapi import FastAPI
import joblib, numpy as np

app = FastAPI()

model = joblib.load('model.pkl')

@app.post('/predict')
def predict(data: dict):
    x = np.array([list(data.values())])
    pred = model.predict(x)[0]
    return {'prediction': float(pred)}
```

Flask (Simple & Lightweight)

```
from flask import Flask, request, jsonify
import joblib, numpy as np

app = Flask(__name__)

model = joblib.load('model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    x = np.array([list(data.values())])
    pred = model.predict(x)[0]
    return jsonify({'prediction': float(pred)})
```

③ Dockerizing the Model

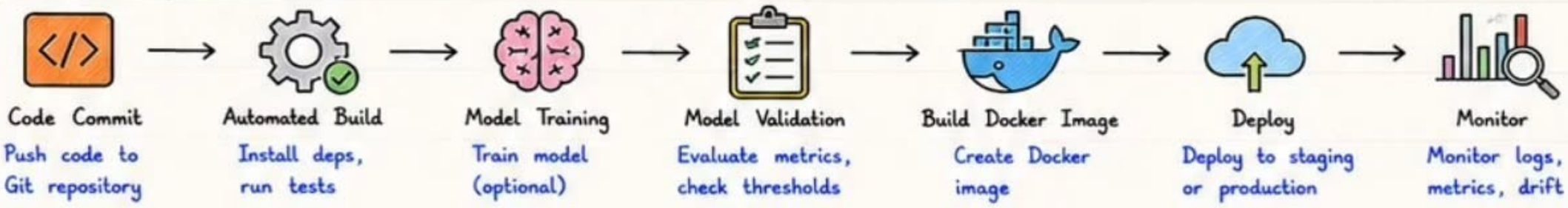
Package the model + code + dependencies into a Docker image for consistent deployment.

```
# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

Run Container

```
docker build -t ml-api .
docker run -p 8000:8000 ml-api
```

④ CI/CD Pipeline for ML



Code Commit	Automated Build	Model Training	Model Validation	Build Docker Image	Deploy	Monitor
Push code to Git repository	Install deps, run tests	Train model (optional)	Evaluate metrics, check thresholds	Create Docker image	Deploy to staging or production	Monitor logs, metrics, drift

⑤ Model Monitoring

What to Monitor?

- ✓ Prediction accuracy / metrics
- ✓ Data drift
- ✓ Model drift
- ✓ Latency & throughput
- ✓ Error rates
- ✓ Business KPI impact

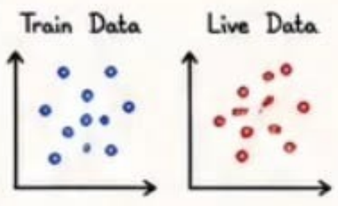
Monitoring Tools

- Prometheus + Grafana
- Evidently AI
- WhyLabs
- MLflow
- Amazon SageMaker Model Monitor

⑥ Data Drift vs Model Drift

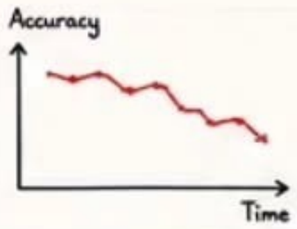
Data Drift

Input data distribution changes over time.



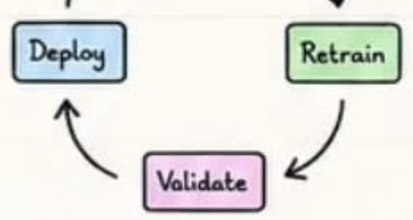
Model Drift

Model performance drops over time.

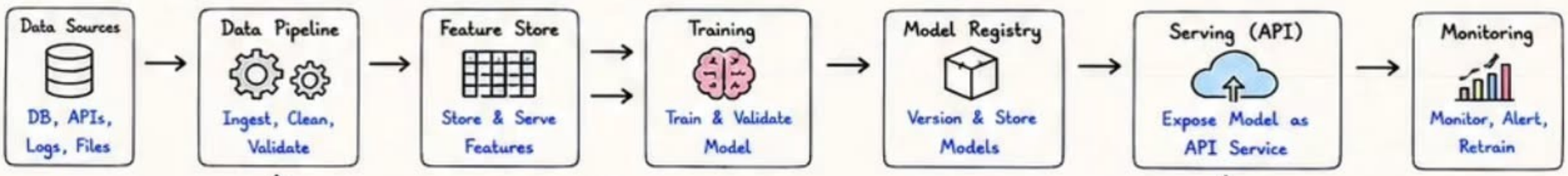


⑦ Retraining Strategy

- ✓ Detect drift or performance drop
- ✓ Collect new data
- ✓ Label (if needed)
- ✓ Retrain model
- ✓ Validate new model
- ✓ Deploy new version
- ✓ Monitor continuously



⑧ MLOps Architecture (High Level)



Key Takeaway

MLOps ensures ML models are reliable, scalable and continuously improving in real-world environments.

Best Practices

- ✓ Automate everything
- ✓ Version your data & models
- ✓ Monitor continuously
- ✓ Start simple, iterate & improve

Interview Facts

- Serialization → joblib, pickle, ONNX
- APIs → FastAPI (async, fast), Flask (simple)
- Docker → Consistent environments
- CI/CD → Automate build, test, deploy
- Drift → Data distribution / performance change

Common Mistakes

- ✗ Not monitoring after deployment
- ✗ Training-serving skew
- ✗ Ignoring data quality
- ✗ No versioning or rollback strategy

ML Quick Revision & Interview Guide

SRIPADH K

One Page To Revise Entire Machine Learning Journey!

1 Complete ML Workflow

1. Business Problem → 2. Data Collection → 3. EDA → 4. Preprocessing → 5. Feature Engineering → 6. Model Training → 7. Evaluation → 8. Tuning → 9. Final Model → 10. Deployment & Monitoring

Iterate & Improve Continuously

2 Algorithm Selection Cheat Sheet

Problem Type	Algorithms	Best For	Notes
Classification	Logistic Regression, DT, Random Forest, SVM, XGBoost, LightGBM	Spam detection, Churn prediction, Fraud detection	Use ROC-AUC, F1 for imbalanced data
Regression	Linear Regression, Ridge, Lasso, SVR, XGBoost, LightGBM	Price prediction, Demand forecasting	Use RMSE, MAE, R^2
Clustering	K-Means, Hierarchical, DBSCAN	Customer segmentation, Grouping similar data	No labels, use silhouette score
Dimensionality Reduction	PCA, t-SNE, UMAP, Autoencoders	Visualization, Remove redundancy	PCA for linear, t-SNE for non-linear
Anomaly Detection	Isolation Forest, One-Class SVM, Local Outlier Factor	Fraud, Network intrusions, Defect detection	Works on normal data patterns
Recommendation	Collaborative Filtering, Content Based, Matrix Factorization	Product recommendation, Movie/Item suggestions	Use implicit/explicit feedback

3 Key Metrics & Formulas

@ qa_insights

A. Classification Metrics

- Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$
- Precision = $\frac{TP}{TP + FP}$
- Recall (Sensitivity) = $\frac{TP}{TP + FN}$
- F1 Score = $2 \times \frac{Precision \times Recall}{Precision + Recall}$
- ROC-AUC = Area Under ROC Curve

Confusion Matrix

Predicted \ Actual	Positive	Negative
Positive	TP	FP
Negative	FN	TN

B. Regression Metrics

- MAE = $\frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$
- MSE = $\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$
- RMSE = $\sqrt{MSE}$
- $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Remember

- ✓ Choose metric based on business goal.
- ✓ Accuracy can be misleading for imbalanced data.
- ✓ Lower MAE/RMSE is better.
- ✓ Higher R^2 & ROC-AUC is better.

4 Common Interview Questions

- What is Machine Learning? Types of ML?
- Supervised vs Unsupervised vs Reinforcement Learning?
- Difference between Parameter and Hyperparameter?
- What is Overfitting and how to handle it?
- Bias vs Variance trade-off?
- How does Gradient Descent work?
- What is Cross-Validation and why use it?
- Difference between Normalization & Standardization?
- Explain Confusion Matrix and metrics.
- When to use K-Means vs DBSCAN?
- What is Regularization? (L1 vs L2)
- What is Ensemble Learning? Types?
- What is PCA and how does it work?
- How would you deploy an ML model?
- How do you handle Data Drift & Model Drift?
- Explain a ML project you have worked on.

5 Project Explanation Framework

1. Problem Statement → What is the business problem and goal?
2. Data Understanding → Sources, size, features, challenges.
3. EDA & Insights → Key patterns, correlations, visual findings.
4. Preprocessing → Missing values, encoding, scaling, outliers.
5. Feature Engineering → New features, selection, transformations.
6. Modeling → Algorithms used and why.
7. Evaluation → Metrics, results, comparison.
8. Tuning → Hyperparameter tuning strategy.
9. Final Model → Best model, performance summary.
10. Deployment Impact → How it helps business, next steps.

6 Important Concepts Checklist

- ✓ Data → Information → Knowledge
- ✓ Train / Validation / Test Split
- ✓ Feature Engineering is key
- ✓ Prevent Data Leakage
- ✓ Right Metric for Right Problem
- ✓ Baseline Model First
- ✓ Iterate & Improve
- ✓ Monitor in Production
- ✓ Retrain when performance drops
- ✓ Business Impact is the final goal

Interview Tip

Always think out loud, explain your assumptions, and justify your choices with both technical + business reasons.

7 Quick Formula / Concept Summary

- Gradient Descent Update:
  $\theta_{new} = \theta_{old} - \alpha \nabla J(\theta)$
  (α = learning rate)
- Sigmoid / Logistic Function:
  $\sigma(z) = \frac{1}{1 + e^{-z}}$
- Regularization (Cost Function):
  $J = MSE + \lambda \sum_{i=1}^{n} w_i^2$ (L2)
  $J = MSE + \lambda \sum_{i=1}^{n} |w|$ (L1)

Overfitting vs Underfitting

Overfitting	Underfitting
• Low bias, High variance	• High bias, Low variance
• High train score	• Low train score
• Low test score	• Low test score
• Complex model	• Too simple model

Best Fit

- Balance bias & variance
- Good performance on train & test

8 Key Takeaway

- ★ Understand the problem deeply.
- ★ Good data + Good features = Strong model.
- ★ Evaluate properly and tune smartly.
- ★ Deploy responsibly and monitor continuously.
- ★ ML is iterative – Learn, Improve, Repeat!

Remember

Models are just tools.
Value comes from solving real business problems!